

School of Information and Physical Sciences
COMP2240/COMP6240 - Operating Systems
Assignment 2 (15%)

Submit using Canvas by **11:59 pm, Friday 27th September 2024**

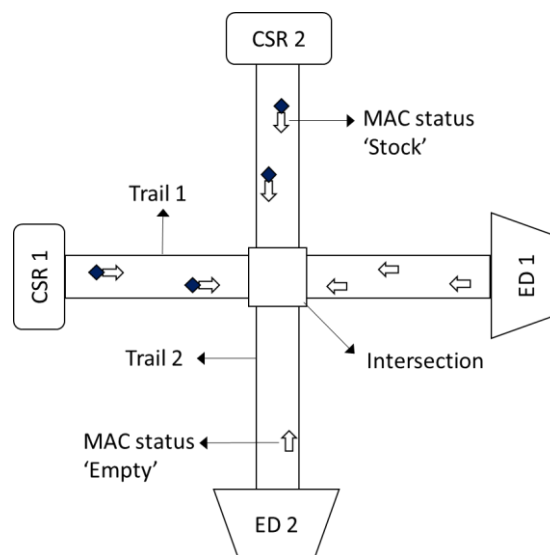
Tasks:

Problem 1, and 2 are both COMP2240 & COMP6240 students.

Problem 3 is only for COMP6240 students.

Problem 1: MAC Management

In a large modern hospital, the two emergency departments (ED1 and ED2) are located in the East and South wing, respectively, and the central supply rooms (CSR1 and CSR2) for those two EDs are located in the West and North wing, respectively. The hospital uses autonomous carts, called MAC (Medical-supply Automated Cart), for transporting medicine, equipment and supplies between the EDs and CSRs. Trail 1 and Trail 2, which connect CSR1 with ED1 and CSR2 with ED2, respectively, intersect with each other. The partial layout of the hospital with two EDs, two CSRs and their trails are shown below.



MACs are simple autonomous carts which iteratively travel back and forth in **the same trail**, carrying 'Stock' in their trip from CSR to ED and return 'Empty' in the return trip. Each MAC has a unique number for identification but its status changes between 'Stock' and 'Empty' depending on the direction of the trip. After initial setup, it was immediately identified that MACs operating in both trails can collide in the intersection, therefore, the hospital employed you to solve the problem. The assigned task is as follows:

1. The intersection can become deadlocked if two MACs from different directions enter the intersection (MACs can only go forward). Collision is evident if more than one MAC enter the intersection simultaneously. So, for safety purpose, not more than one MAC is allowed to enter the intersection.
2. The solution should be starvation free. I.e. a stream of MACs from any one direction should not prevent MACs from other directions to cross the intersection.

3. There are three checkpoints (sensors) in the intersection to which each MAC reports. Add 50 ms delay after each checkpoint to simulate the time to pass the checkpoint. The system keeps track of MACs passing in both trails (i.e. counts every time a MAC in a trail crosses the intersection).
4. Loading and unloading of supplies happen instantly (i.e. the assumption is no time is required) after a MAC crosses the intersection.

Using **semaphores**, design and implement an algorithm that prevents deadlock. Use **threads** to simulate **multiple concurrent** MACs and assume that the group of MACs are iteratively attempting to use the intersection from all four directions, i.e. $CSR1 \rightarrow ED1$, $ED1 \rightarrow CSR1$, $CSR2 \rightarrow ED2$ and $ED2 \rightarrow CSR2$.

Your program should input parameters at runtime to initialise the number of MACs from each direction and the number of intersection crossings for MACs. For example, the input “ $CSR1=1, CSR2=2, ED1=1, ED2=1, N=2$ ” indicates that the simulation should start with 1, 2, 1 and 1 MACs starting from CSR1, CSR2, ED1 and ED2, respectively, and each of the MACs should stop after crossing the intersection 2 times. MACs should be numbered continuously (i.e. as MAC-1, MAC-2 etc.) starting from CSR1 then CSR2, ED1 and ED2. Depending on whether a MAC is going towards the ED or towards the CSR it’s status will be “Stock” or “Empty”, respectively.

Sample Input/output for Problem 1.

Input: Your program will read all information from an input file, e.g., `P1-1.txt` (taken via command line argument).

The input will be as follows:

`CSR1=1, CSR2=2, ED1=1, ED2=1, N=2`

The input indicates that the program is initialized with 1 ‘Stock’ carrying MAC from CSR1, 2 ‘Stock’ carrying MACs from CSR2, 1 ‘Empty’ MAC from ED1 and 1 ‘Empty’ MAC from ED2 which will cross the intersection for two times ($N=2$) going between CSRs and EDs and changing status accordingly. The MACs will be numbered as follows, from CSR1 (MAC-1), from CSR2 (MAC-2 and MAC3), from ED1 (MAC-4) and from ED2 (MAC-5).

Output: The output from one execution is as follows:

```
MAC-1 (Stock): Waiting at the Intersection. Going towards ED1
MAC-4 (Empty): Waiting at the Intersection. Going towards CSR1
MAC-5 (Empty): Waiting at the Intersection. Going towards CSR2
MAC-3 (Stock): Waiting at the Intersection. Going towards ED2
MAC-2 (Stock): Waiting at the Intersection. Going towards ED2
MAC-1 (Stock): Crossing intersection Checkpoint 1.
MAC-1 (Stock): Crossing intersection Checkpoint 2.
MAC-1 (Stock): Crossing intersection Checkpoint 3.
MAC-1 (Stock): Crossed the intersection.
Total crossed in Trail1: 1 Trail2: 0
MAC-4 (Empty): Crossing intersection Checkpoint 1.
MAC-1 (Empty): Waiting at the Intersection. Going towards CSR1
MAC-4 (Empty): Crossing intersection Checkpoint 2.
MAC-4 (Empty): Crossing intersection Checkpoint 3.
MAC-4 (Empty): Crossed the intersection.
Total crossed in Trail1: 2 Trail2: 0
MAC-3 (Stock): Crossing intersection Checkpoint 1.
MAC-3 (Stock): Crossing intersection Checkpoint 2.
MAC-4 (Stock): Waiting at the Intersection. Going towards ED1
MAC-3 (Stock): Crossing intersection Checkpoint 3.
MAC-3 (Stock): Crossed the intersection.
Total crossed in Trail1: 2 Trail2: 1
MAC-5 (Empty): Crossing intersection Checkpoint 1.
MAC-3 (Empty): Waiting at the Intersection. Going towards CSR2
MAC-5 (Empty): Crossing intersection Checkpoint 2.
MAC-5 (Empty): Crossing intersection Checkpoint 3.
MAC-5 (Empty): Crossed the intersection.
Total crossed in Trail1: 2 Trail2: 2
MAC-2 (Stock): Crossing intersection Checkpoint 1.
MAC-2 (Stock): Crossing intersection Checkpoint 2.
MAC-5 (Stock): Waiting at the Intersection. Going towards ED2
MAC-2 (Stock): Crossing intersection Checkpoint 3.
MAC-2 (Stock): Crossed the intersection.
Total crossed in Trail1: 2 Trail2: 3
MAC-1 (Empty): Crossing intersection Checkpoint 1.
MAC-1 (Empty): Crossing intersection Checkpoint 2.
MAC-2 (Empty): Waiting at the Intersection. Going towards CSR2
MAC-1 (Empty): Crossing intersection Checkpoint 3.
MAC-1 (Empty): Crossed the intersection.
Total crossed in Trail1: 3 Trail2: 3
MAC-1: Finished.
MAC-4 (Stock): Crossing intersection Checkpoint 1.
MAC-4 (Stock): Crossing intersection Checkpoint 2.
MAC-4 (Stock): Crossing intersection Checkpoint 3.
MAC-4 (Stock): Crossed the intersection.
Total crossed in Trail1: 4 Trail2: 3
MAC-4: Finished.
MAC-3 (Empty): Crossing intersection Checkpoint 1.
MAC-3 (Empty): Crossing intersection Checkpoint 2.
MAC-3 (Empty): Crossing intersection Checkpoint 3.
MAC-3 (Empty): Crossed the intersection.
Total crossed in Trail1: 4 Trail2: 4
MAC-3: Finished.
MAC-5 (Stock): Crossing intersection Checkpoint 1.
MAC-5 (Stock): Crossing intersection Checkpoint 2.
MAC-5 (Stock): Crossing intersection Checkpoint 3.
MAC-5 (Stock): Crossed the intersection.
Total crossed in Trail1: 4 Trail2: 5
MAC-5: Finished.
MAC-2 (Empty): Crossing intersection Checkpoint 1.
MAC-2 (Empty): Crossing intersection Checkpoint 2.
MAC-2 (Empty): Crossing intersection Checkpoint 3.
MAC-2 (Empty): Crossed the intersection.
Total crossed in Trail1: 4 Trail2: 6
MAC-2: Finished.
```

Remark: For the same input the output may look somewhat different from run to run.

Problem 2: Hot or Iced coffee?

School of Information and Physical Sciences (SIPS), UON bought a coffee machine that can serve both iced and hot coffee for the staffs. The coffee machine has three dispensing heads which can serve three clients (staffs) in parallel. We call a staff Hot Client (H) if (s)he is looking for hot coffee and a Cold Client (C) if (s)he is looking for cold coffee. However, the machine can operate in either of its two modes (hot or cold drink) at a time. If a Hot Client has started to make coffee in the machine then the other two vacant dispersers can serve hot coffee only – a Cold Client must wait. A client (Hot or Cold) can choose the brew strength by choosing the brew time at the beginning. So the assumptions in operating the coffee machine are

- Hot and Cold clients cannot use the machine at the same time.
- No more than three clients can use the machine simultaneously.
- Each client can choose different brew strength for her/his coffee and will take different time to brew her/his coffee.
- A Hot client with ID y (i.e. H_y) should NOT be served before a Hot client with ID x (i.e. H_x) where $x < y$. And the same for the Cold clients.

Using **monitor**, design and implement an algorithm that ensures the operation of the coffee machine according to the above characteristics. Use **threads** to simulate **multiple concurrent** clients. Your solution should be fair – stream of Hot Clients should not prevent a Cold Client from using the coffee machine or vice versa.

Sample Input/output for Problem 2.

Input: Your program will read all information from an input file, e.g., `P2-1.txt` (taken via command line argument).

The input will be as follows:

```
9
H1 4
H2 5
H3 3
C1 5
C2 3
C3 2
C4 2
H4 3
H5 2
```

Where the first line contains the number of clients looking for coffee and each subsequent line contains information about each client in the following form

Client-ID Brew-Time

Client-ID: The first character is H or C indicating hot (H) or cold (C) coffee client, a number (without no blank in-between) indicating the client ID in each group. Client-IDs are unique.

Brew-Time: The time to brew his coffee.

The order of clients in the input file indicates the order in which they arrived for coffee.

Output: The output should be as follows:

```
(0) H1 uses dispenser 1 (time: 4)
(0) H2 uses dispenser 2 (time: 5)
(0) H3 uses dispenser 3 (time: 3)
(5) C1 uses dispenser 1 (time: 5)
(5) C2 uses dispenser 2 (time: 3)
(5) C3 uses dispenser 3 (time: 2)
(7) C4 uses dispenser 3 (time: 2)
(10) H4 uses dispenser 1 (time: 3)
(10) H5 uses dispenser 2 (time: 2)
(13) DONE
```

Each line contains information about the usage of the coffee machine by a client. First, the time the client starts using the coffee machine in parenthesis. Then his/her Client-ID, the dispenser number in which she/he is operating and her/his brew time in parenthesis.
Last line shows the time to serve all the clients.

Remark: If more than one client are dispatched at the same time, then those may be assigned to different dispensers from run to run.

Problem 3: Additional Task For COMP6240 Students Only

[NOT for COMP2240 students]

In addition to the above simulations, you need to write a report on the software approaches to mutual exclusion.

Objective: In lecture, two algorithms, namely *Dekker's algorithm* and *Peterson's algorithm*, were discussed as software approaches to mutual exclusion. A couple of other algorithms exist in the literature for the same purpose. The objective of this assignment is to conduct a detailed research and analysis on the various software approaches to mutual exclusion.

Description: You are tasked to perform a survey on the software approaches to mutual exclusion and prepare a report on that. Your survey should include at least four different algorithms (*may or may not include the above two*). You should discuss the design principle, suitability of generalisation for n processes, suitability for multiprocessor/multi-core environment, relative advantages/disadvantages.

Report Structure and Length: Your report should contain the following sections (i) Introduction (ii) Software approaches to Mutual Exclusion (iii) Comparative Analysis (iv) Conclusion and (v) References. The length of the report, covering all the sections mentioned before should be 6~10 pages (A4). The report should be written in a formal academic style, with proper citations and references.

The report should be prepared based on your own research and any kind of plagiarism including use of Generative AI in preparing this report should be considered as a serious breach of academic integrity.

1. Submission Requirements:

Your submission must also conform to the follow considerations.

1.1. Programming Language:

The programming language is Java, versioned as per the University Lab Environment (*Note this is currently a **subversion of Java 17***). You may only use standard Java libraries as part of your submission.

1.2. User Interface:

The output should be printed to the console, and strictly following the output samples given in the assignment package. While there are no marks allocated specifically for the Output Format, **there will be a deduction when the result values and result format vary** from those provided.

1.3. Input and Output:

Your program will accept data from an input file of name specified as a command line argument. The sample files `P1-1.txt` and `P2-1.txt` (*containing inputs for Problem 1 and 2 respectively*) are provided to demonstrate the required input file format. **Hint:** the **Java Scanner Library** is something you will likely want to use here!

Your submission will be tested with the above data and will also be tested with other input files.

Your program should output to standard output (*this means output to the Console*). Output should be strictly in the given format (*see the output files below*).

The sample files `P1-1out.txt` and `P2-1out.txt` (*containing output for `P1-1.txt` and `P2-1.txt` respectively*) are provided to demonstrate the required output (and input) format which **must be strictly maintained**.

If output is not generated in the required format then your program will be considered incorrect.

1.4. Use of Generative AI

For completing the programming component of this assignment, you are encouraged to use Generative AI (GenAI) tools. **You must not use GenAI in preparing the reports.** Please check the document “Use of Generative AI in Assignment 2” for information, explanation and possible use cases on how such tools to be used in preparing the solution before you start working on this assignment.

1.5. Mark Distribution:

Mark distribution can be found in the assignment feedback document (`Assign2Feedback2240.pdf` and `Assign2Feedback6240.pdf`).

1.6. Deliverable:

1. Your submission will contain your program source code for all problems, with documentation and the report (*below*) in the root of the submission. These files will be zipped and submitted in an archive named **`c9876543.zip`** (*where `c9876543` is your student number*) – do not submit a `.rar`, or a `.7z`, or etc.
2. Your **main classes** for different problems should be **`P1.java`**, and **`P2.java`** and your program will compile with the command lines **`javac P1.java`**, and **`javac P2.java`** respectively. Your

program will be executed by running `java P1 input.txt, j` and `java P2 input.txt` respectively.

...where `input.txt` can be *any* filename – **do not hardcode filenames or extensions!**

Note: If your program can not be compiled and executed in the expected manner (*above*) then please add a `readme.txt` (containing any special instructions required to compile and run the source code) in the root of the submission.

Please note that such non-standard submissions will be marked with heavy penalty.

3. You should write a 2 page (A4) reflective report (name as **Report.pdf**) on the use of Generative AI (GenAI) in completing this assignment. In your reflective note, you should provide a detailed account of your experience and should include, but not limited to, the following elements – (i) overview of the tool(s) used (ii) how the tool(s) was/were integrated in your workflow (iii) what was the quality/correctness of the response generated by the tool(s) and/or how the GenAI tool(s) improved the quality of your solution (iv) how the tool(s) improved your learning experience and problem solving skills (v) what challenges you faced in using those tools and how you solved those challenges (vi) how useful was the tool for your productivity e.g. how much time was saved by using GenAI tool(s) (vii) any ethical consideration or concerns in using GenAI tools in this assessment etc. **GenAI must not be used in preparing this report.**

If you did not use GenAI in preparing this assignment at all then:

You should write a brief **1 page** (A4) report of the how you tested your programs to ensure they enforced mutual exclusion and are deadlock and starvation free. Specifically, your report should include discussion on edge cases you considered and behaviour of your algorithm on those cases, any specific trick/technique you applied and did you face any specific issue.

4. **[COMP6240 students only]** You should submit the research report (name as **Research_Report.pdf**) on the task in Section 3. **GenAI must not be used in preparing this report.**

NOTES:

- a) Assignments submitted after the deadline (**11:59 pm 27th September 2024**) will have the maximum marks available reduced by 10% per 24 hours.
- b) If your assignment is not prepared and submitted following above instructions then you will lose most of your marks despite your algorithms being correct.
- c) If you have been granted an extension, please include a copy of the extension approval in as a **PDF document**, called **Extension.pdf**, and place this in the root of your submission, along with your other documents and code.